



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №2
Технології розробки програмного забезпечення
«ДІАГРАМА ВАРІАНТІВ ВИКОРИСТАННЯ. СЦЕНАРІЇ ВАРІАНТІВ
ВИКОРИСТАННЯ. ДІАГРАМИ UML. ДІАГРАМИ КЛАСІВ.
КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ»

Виконав:
студент групи ІА-24
Іскандаров Е. Е.
Перевірив:
Мягкий М. Ю.

Київ 2024

Зміст

Зміст	2
Короткі теоретичні відомості.....	3
Хід роботи	5
Діаграми варіантів використання (Use-case diagram)	6
UML діаграма класів.....	10
Посилання на репозиторій:	13
Висновок	13

Тема: ДІАГРАМА ВАРІАНТІВ ВИКОРИСТАННЯ. СЦЕНАРІЇ ВАРІАНТІВ ВИКОРИСТАННЯ. ДІАГРАМИ UML. ДІАГРАМИ КЛАСІВ. КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ

Мета: Розробити діаграми варіантів використання та класів.

Короткі теоретичні відомості

Діаграма варіантів використання (Use Case Diagram)

Діаграма варіантів використання описує функціональні можливості системи з точки зору її користувачів (акторів).

- **Елементи:**
 - **Актори:** користувачі системи (люди, пристрої чи інші системи).
 - **Варіанти використання (Use Cases):** функції чи послуги, які система надає акторам.
 - **Зв'язки:** відображають взаємодію акторів із варіантами використання.
- **Призначення:** допомагає зрозуміти, що система повинна робити, і визначити її функціональність.

Сценарії варіантів використання (Use Case Scenarios)

Сценарій варіанту використання описує конкретну послідовність дій між актором і системою для досягнення певної цілі.

- **Основні компоненти:**
 - Назва сценарію.
 - Опис: короткий огляд того, що відбувається.
 - Основний потік: стандартна послідовність дій.
 - Альтернативні потоки: відхилення від основного сценарію.
 - Попередні умови: що має бути виконано перед початком сценарію.
 - Результат: стан системи після виконання сценарію.

Діаграми UML (Unified Modeling Language)

UML — це мова моделювання для створення візуальних діаграм, які описують різні аспекти системи.

- **Основні типи діаграм:**

1. **Структурні:** діаграми класів, компонентів, об'єктів, розгортання.
2. **Поведінкові:** діаграми варіантів використання, активності, послідовності, станів.
3. **Інтеракційні:** діаграми комунікації, часові діаграми.

- **Призначення:** полегшення аналізу, дизайну та документування системи.

Діаграма класів (Class Diagram)

Діаграма класів моделює структуру системи, відображаючи класи, їх атрибути, методи та взаємозв'язки.

- **Елементи:**
 - **Класи:** описують об'єкти (атрибути та методи).
 - **Зв'язки:**
 - Асоціація (association).
 - Агрегація (aggregation).
 - Композиція (composition).
 - Наслідування (inheritance).
 - **Мультиплікатори:** показують кількість об'єктів у зв'язку.
- **Призначення:** деталізує статичну структуру системи.

Концептуальна модель системи

Концептуальна модель системи — це абстрактне уявлення про основні об'єкти домену і зв'язки між ними.

- **Компоненти:**
 - Об'єкти/класи: реальні чи абстрактні сутності, що мають атрибути та операції.
 - Атрибути: властивості об'єктів.
 - Зв'язки: взаємодії між об'єктами.
- **Призначення:** допомагає зрозуміти бізнес-логіку та основні взаємозв'язки в системі.
- **Застосування:** використовується на початкових етапах проєктування для формування загального уявлення про систему.

Хід роботи

Web-browser (proxy, chain of responsibility, factory method, template method, visitor, p2p) Веб-браузер повинен мати можливість зробити наступне: мати адресний рядок для введення адреси сайту, переміщатися і відображати структуру html документа, переглядати підключений javascript та css файли, перегляд всіх підключених ресурсів (зображень), коректна обробка відповідей з сервера (коди відповідей HTTP) - переходи при перенаправленнях, відображення сторінок 404 і 502/503.

Діаграми варіантів використання (Use-case diagram)

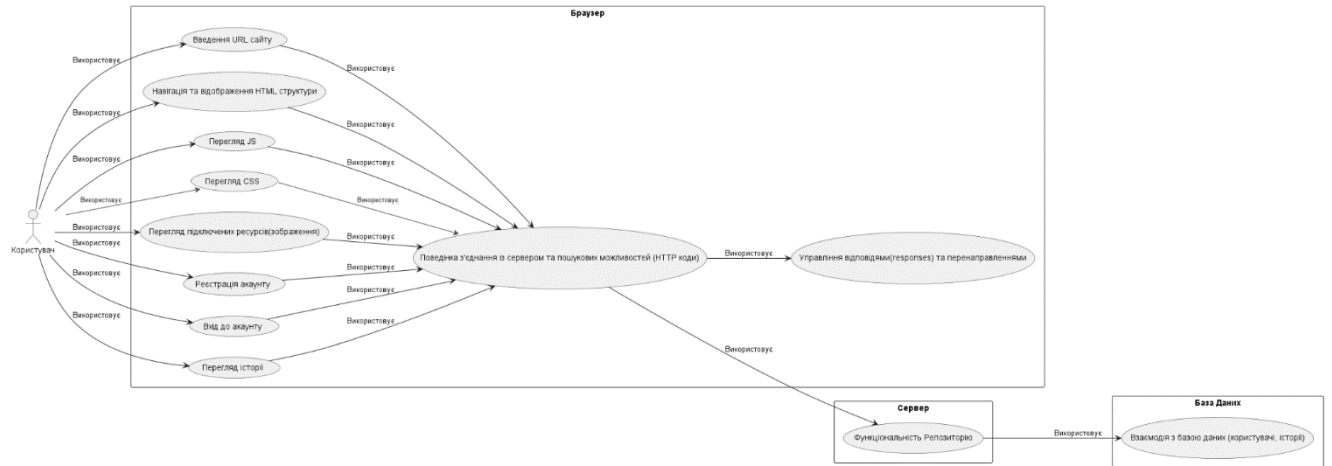


Рис. 1.1 - Use-case diagram

На діаграмі випадків використання (Use Case Diagrams), які ілюструють функціональність веб-браузера. Діаграма деталізує функції, доступні для звичайного користувача, що дозволяє краще розуміти процеси користування веб-браузером.

Функціональність для звичайного користувача

На діаграмі, що описує взаємодію користувача з браузером, основними випадками використання є:

Навігація та доступ до сайту:

- Введення URL сайту: дозволяє користувачу переходити на веб-ресурс
- Навігація та відображення HTML структури: забезпечує перегляд та взаємодію з веб-сторінками
- Перегляд JS та CSS: надає доступ до інтерактивних елементів та стилів сторінки

Авторизація та реєстрація:

- Реєстрація акаунту: створення нового облікового запису в системі
- Вхід до акаунту: автентифікація користувача для доступу до персоналізованого контенту

Робота з контентом:

- Перегляд статей: доступ до текстового та мультимедійного контенту
- Перегляд локального ресурсу/збереження: можливість працювати з локально збереженими даними

Взаємодія з сервером та базою даних

Серверна частина:

- Реалізація взаємодії із сервером та поданням мультимедіа (HTTP код): забезпечує обмін даними між клієнтом та сервером
- Управління колекціями та переключеннями: контролює відображення та організацію контенту
- Функціональність Репозиторію: керує збереженням та доступом до даних

База даних:

- Взаємодія з базою даних: зберігає інформацію про користувачів та контент

Типи зв'язків на діаграмі:

1. **Пряма взаємодія:** показує безпосередній зв'язок між користувачем та функціями браузера
2. **Послідовна взаємодія:** демонструє потік даних від браузера через сервер до бази даних

Загальний висновок

Діаграма відображає повний цикл взаємодії користувача з веб-системою, починаючи від введення URL і закінчуючи збереженням даних. Архітектура забезпечує логічний розподіл функціональності між браузером, сервером та базою даних, що створює ефективну та масштабовану систему для роботи з веб-контентом.

Варіанти використання (прецеденти)

Прецедент 1: Пошук сторінки за її адресою

- Опис: Користувач здійснює пошук сторінки.
- Актори: Користувач.
- Умови успішності: Користувач отримав виведену сторінку.
- Основний сценарій:
 - Користувач запускає веб-браузер.
 - Користувач ініціалізує пошук сторінки.
 - Веб-браузер проводить пошук сторінки.
 - Після завершення пошуку, браузер виводить користувачу результат(необхідну сторінку або ж сторінку помилки).

Прецедент 2: Вхід до акаунту

- Опис: Користувач намагається увійти до свого акаунту.
- Актори: Користувач, Веб-браузер.
- Умови успішності: Користувач зареєстрований.
- Основний сценарій:
 - Користувач натискає на кнопку увійти до акаунту.
 - Користувач вводить свої дані.
 - Веб-браузер проводить обробку запиту, перевіряючи чи цей користувач зареєстрований
 - Веб-браузер повертає результат у вигляді повідомлення про успішний чи ні вхід до акаунту(в залежності, чи цей користувач зареєстрований)

Прецедент 3: Перегляд історії

- Опис: Зареєстрований користувач переглядає історію відвідувань.
- Актори: Зареєстрований користувач або Адміністратор.
- Умови успішності: Зареєстрований користувач або адміністратор переглядають історію відвідувань.
- Основний сценарій:
 - Зареєстрований користувач або адміністратор натискає кнопку перегляду історії.
 - Веб-сервер приймає запит та обробляє його.
 - В залежності від того, чи це зареєстрований користувач або адміністратор виводить результат у вигляді історії зареєстрованого користувача(у випадку зареєстрованого користувача) або історій всіх користувачів(у випадку адміністратора)

UML діаграма класів

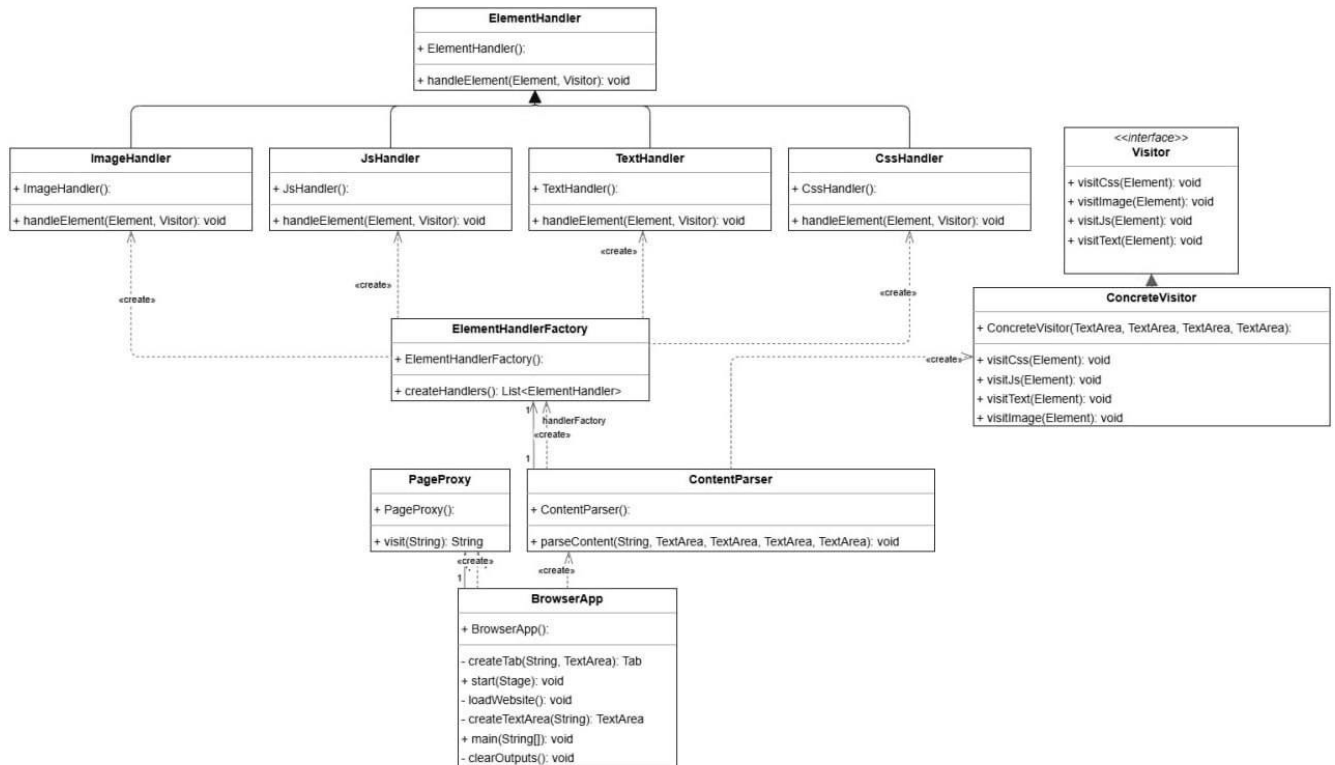


Рис. 2.1 – UML діаграма класів

Діаграма відображає основні класи системи, їх властивості, методи та зв'язки, які дозволяють організувати обробку елементів HTML, CSS, JavaScript, тексту та зображень.

Основні класи системи:

1. ElementHandler (Базовий обробник елементів)

Поля: `ElementHandler()`

Методи: `handleElement(Element, Visitor): void` — метод для обробки елемента.

Зв'язки: Є базовим класом для обробників різних типів елементів: **ImageHandler**, **JsHandler**, **TextHandler**, **CssHandler**.

2. ImageHandler (Обробник зображень)

Наслідує **ElementHandler**.

Методи: `handleElement(Element, Visitor): void` — обробляє зображення.

3. JsHandler (Обробник JavaScript)

Наслідує **ElementHandler**.

Методи: `handleElement(Element, Visitor): void` — обробляє JavaScript.

4. TextHandler (Обробник тексту)

Наслідує `ElementHandler`.

Методи: `handleElement(Element, Visitor): void` — обробляє текст.

5. CssHandler (Обробник CSS)

Наслідує `ElementHandler`.

Методи: `handleElement(Element, Visitor): void` — обробляє CSS.

6. ElementHandlerFactory (Фабрика обробників елементів)

Поля: `ElementHandlerFactory()`

Методи: `createHandlers(): List<ElementHandler>` — створює список обробників елементів.

Зв'язки: Створює об'єкти `ImageHandler`, `JsHandler`, `TextHandler`, `CssHandler`.

7. Visitor (Інтерфейс відвідувача)

Методи:

`visitCss(Element): void` — обробка CSS.

`visitImage(Element): void` — обробка зображення.

`visitJs(Element): void` — обробка JavaScript.

`visitText(Element): void` — обробка тексту.

8. ConcreteVisitor (Конкретний відвідувач)

Наслідує інтерфейс `Visitor`.

Поля: `ConcreteVisitor(TextArea, TextArea, TextArea, TextArea)` — параметри для збереження результатів обробки.

Методи: Реалізує методи `visitCss`, `visitImage`, `visitJs`, `visitText`.

9. PageProxy (Проксі сторінки)

Поля: `PageProxy()`

Методи: `visit(String): String` — відвідування сторінки.

10. ContentParser (Аналізатор контенту)

Поля: `ContentParser()`

Методи: `parseContent(String, TextArea, TextArea, TextArea, TextArea): void` — аналізує контент веб-сторінки.

11. BrowserApp (Додаток браузера)

Поля: `BrowserApp()`

Методи: `createTab(String, TextArea): Tab` — створює вкладку.

`start(Stage): void` — запуск браузера.

`loadWebsite(): void` — завантаження веб-сайту.

`createTextArea(String): TextArea` — створення текстової області.

`main(String[]): void` — головний метод.

`clearOutputs(): void` — очищення виведення результатів.

Зв'язки між класами:

- `ElementHandlerFactory` створює об'єкти обробників: `ImageHandler`, `JsHandler`, `TextHandler`, `CssHandler`.
- `BrowserApp` працює із вкладками через методи створення та очищення.
- `ConcreteVisitor` реалізує логіку обробки елементів, використовуючи методи `visitCss`, `visitImage`, `visitJs`, `visitText`.

Посилання на репозиторій:

https://github.com/Elmir2/trpz_labs

Висновок: У процесі розробки системи управління проектами використано різноманітні засоби моделювання для забезпечення чіткого розуміння структури та функціональності системи:

Діаграми варіантів використання допомагають ідентифікувати ключові ролі користувачів (наприклад, менеджери, розробники) і їх взаємодію із системою. Вони визначають основні функції роботи з веб-браузером.

Сценарії варіантів використання деталізують поведінку системи в рамках кожного випадку використання. Це дозволяє краще зрозуміти послідовність дій користувача та відповідь системи, що сприяє виявленню можливих проблем на етапі проектування.

Діаграми UML надають графічне представлення структури та динаміки системи, зокрема її класів, об'єктів та їх взаємодії. Вони є важливими інструментами для комунікації між членами команди.

Діаграми класів формують основу концептуальної моделі системи, описуючи її основні елементи (класи, атрибути, методи) та зв'язки між ними. Вони забезпечують цілісне уявлення про структуру даних і функціональні можливості системи.

Концептуальна модель системи узагальнює всі ключові аспекти функціональності, забезпечуючи основу для подальшого детального проектування і розробки. Вона дозволяє зберегти баланс між простотою та функціональністю, враховуючи потреби кінцевих користувачів.

Ці засоби моделювання в комплексі забезпечують послідовне, структуроване та прозоре проектування системи, мінімізують ризики помилок та сприяють її успішній реалізації.